HomeWork 2 Laboratorio Bioimmagini

Luca Perrone

15 Dicembre 2023

1 Exercise 1

1.1 Robinson Compass

After observing the input image, the first step in this exercise was to perform Gaussian filtering to remove Gaussian noise. Additionally, a cropping function was applied to eliminate artifacts resulting from the acquisition technique.

Following Gaussian filtering, was conducted to remove noise from image. Subsequently, Robinson compass function was implemented. The output of this has been binarized with a fixed threshold so that ignores non relevant low intensity edges.

The edges detected using this function resulted very thick, so a thinning operation was performed using *bwmorph()* along with a very mild opening to eliminate the salt noise.

To eliminate the hands countours and artifacts from the detected image i first turned darkest pixels as lighter to blend them with the rest of the background, then a mask a mask was created with a fixed threshold to saparate the whole hand from background. So, through the use of *imsubtract()*, was possible to subtract the Robinson Compass output to this mask and return only the bones countours.

Also a gradient filtering was firstly applied to original image for preprocessing edges enanchement but has been decided to remove it for the poor results. (The edges seemed to split in two even with very low structuring element operations)

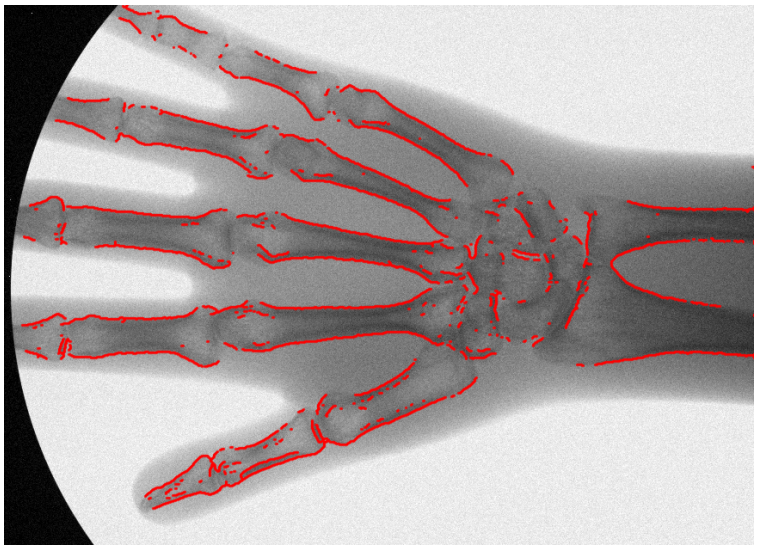

Figure 1: Robinson Compass

1.2 Sobel and Prewitt

In this case, the contrast has been enhanced using an adaptive histogram equalization (*adapthisteq()*) function applied on the original filtered image. This function proved to be highly effective for enhancing the contrast of the image. However, for the Robinson Compass, was chosen to not use it due to the compass's sensitivity to noise introducted.

Sobel and Prewitt detection has been performed using matlab function *edge()*. To remove hands countours, the same process as for Robinson Compass was used.

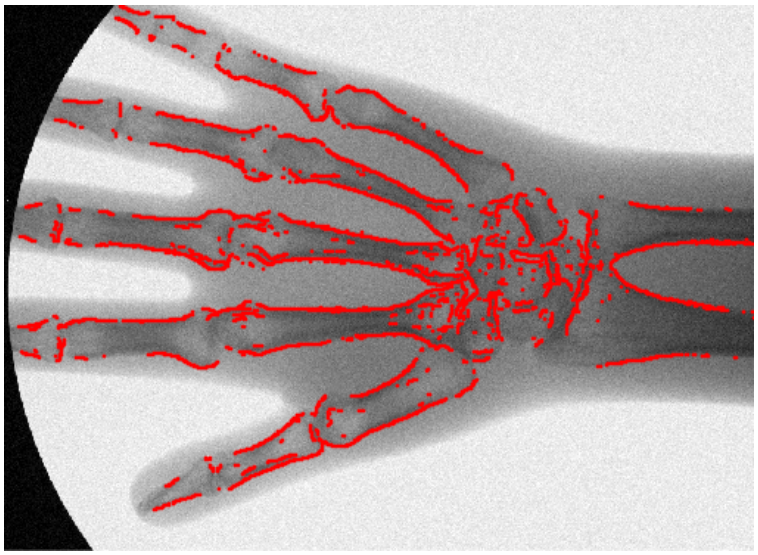

Figure 2: Sobel operator

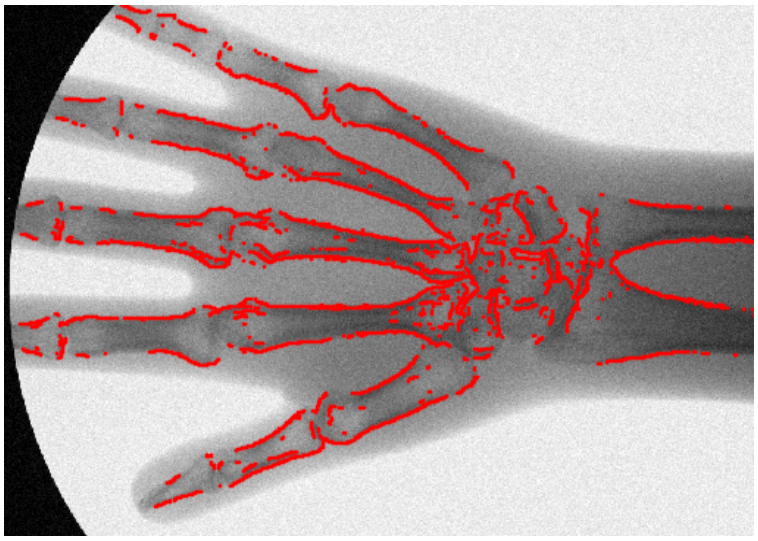

Figure 3: Prewitt operator

2 Exercise 2

2.0.1 Detect the external contours of the blister and compute its aspect ratio

After converting the image to grayscale, the detection of external contours of the blister involved performing binarization using the *imbinarize()* function and *graythresh()* to obtain the threshold value. To address noise outside the blister, the decision was made to erode the image using the *imerode()* function with a 3x3 structuring element.

To calculate the aspect ratio, the *regionprops()* function was employed to retrieve the sizes of the blister bounding box. This function returns a four-element array: the first two are the coordinates of the top-left corner, the third is the horizontal width of the object, and the last one is the vertical width. The aspect ratio can be computed as the product of the largest value between the third and fourth outputs divided by the minimum.

Aspect Ratio: 1.3723

2.0.2 Remove the different illumination effect

Illumination effects were addressed through multiple steps:

- An opening function, *imopen()*, was employed to mitigate light effects.
- A custom function was implemented to attenuate pixels with intensities higher than 0.7 of a fixed value. This divider value increase according to pixel values.
- A subsequent weaker opening was performed to correct noise introduced by the custom attenuation method.
- Attempts were made to implement high-pass or homomorphic filtering to remove illumination effects. However, the results were found to be less effective compared to the used method.

Figure 4: External countours of the blister

Figure 5: Illumination effects removed

2.0.3 Detect the pills still in the blister and the missing ones

Building upon the image obtained in previous sections, both *imadjust()* and *adapthisteq()* were utilized for contrast adjustment. Subsequently, a gradient filter was applied to emphasize only the edges in the image.

The resulting image was binarized and subjected to an opening operation to eliminate salt noise. Additionally, skeletization was performed using *bwskel()* to thin the edges.

To separate the two top pills from the others, *bwfill()* was applied, followed by a robust erosion (55x55) and a milder dilation (40x40). The erosion effectively removed noise and non-relevant edges, while the dilation expanded the obtained pill masks.

To eliminate the blister contours, an eroded version of the previously computed mask from step 1 was used. Pixels corresponding to the blister contours were set to zero in the new binary mask.

For detecting pills, the process began with a thorough analysis of the image to identify suitable tools. It was observed that the HSV colorspace effectively highlighted the pills still in the blister.

So to identify the pills in the blister, the colormap was changed to HSV, and the resulting image was binarized using a fixed threshold of 0.2. This operation generated an inverted mask, which was further adjusted using a **for** loop switch pixel values. A closing function was then applied to remove the remaining noise.

The final step involved subtracting the previously computed mask from this second one using the *imsubtract()* function followed by an opening operation to eliminate errors in the subtraction (i.e the same pill in the two masks has different dimensions). The subtraction resulted in some out-of-range values (-1) due to the binary nature of the images. To rectify this, a **for** loop was employed to set negative values to 0 and positive values to 1.

(a) Total spots for pills in the blister

(b) Pills present in the blister

(c) Missing pills

3 Exercise 3

For this exercise, the initial step involved saving the different 25 slices, converted to double type, in a cell array. Additionally, a histogram equalization was performed to enhance the contrast.

The algorithm comprises a single interaction: the user is prompted to click with the cursor on a pixel within the LV (Left Ventricle) area and press Enter. This interaction allows for storing both the coordinates and the pixel value in a variable.

This was possible because after a first analysis of the slices, LV was almost everytime in the same position, this algoritm couldn't work on a more dynamic model.

At this point, a **for** loop was implemented to analyze each of the 25 slices. A **custom function** for performing k-means was utilized. This function, given an image and a set of coordinates, returned a binary image highlighting only the object overlapping the provided coordinates. The *bwselect()* function was employed to perform this task, and the *bwfill()* function was used to include papillary muscles in the created mask.

The LV area was calculated by summing the white pixels in the mask and converting to milliliters using the resolution values given in the input file, following the formula:

$$PixelSize = \frac{1}{SpatialResolution} = \frac{ImageSize(mm)}{MatrixSIze(pixels)};$$

The eccentricity plot was generated using the *regionprops()* function, and the boundaries of the LV were obtained with the *bwboundaries()* function. Both functions were applied to the binary images of the LV surface computed previously. It was observed that the *bwboundaries()* function stored the coordinates in a clockwise direction. To address this, the *flip()* function was employed to make it counter-clockwise.

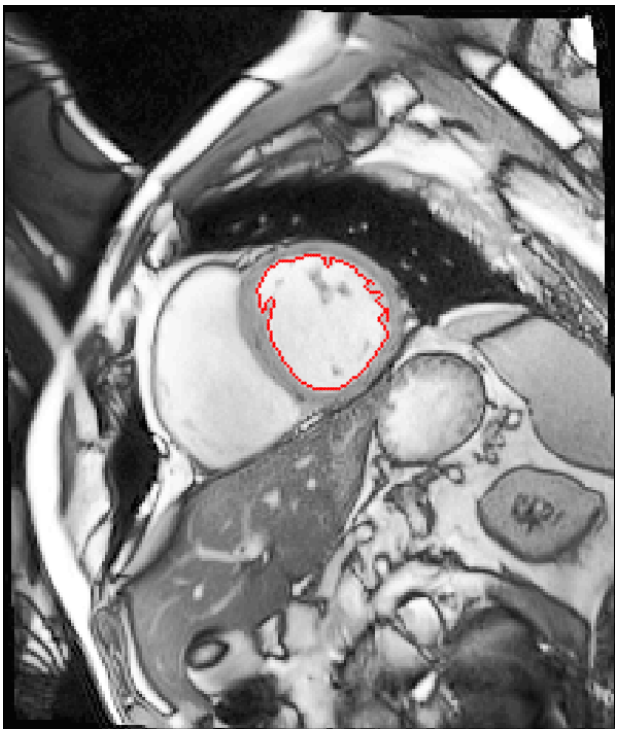

Figure 7: LV countours on first frame

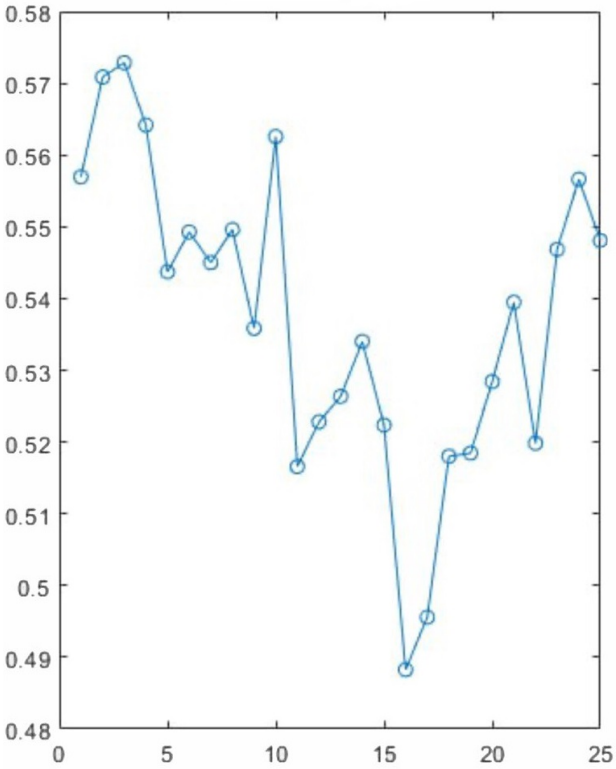

Figure 8: Eccentricity plot over time